3 Assignment 3

3.1 Task 1

In the following code the path from `start` to `end` is going to be searched with a depth-first algorithm. The `graph` passed to the function is an edge list, represented by a dictionary where the key is the vertex and its value is a list of the vertices it is connected to.

Pseudocode

```
1  Define dfs(graph, start, end)
2      Create the stack to track nodes and their paths
3      Put the start node in the stack with itself as the first node of the path
4      While there are values in the stack
5          Pop the stack, returning the starting node and the path
6          If the most recently extracted node is the end node, then return the
               path that was just returned
7          For every neighbour of the most recently returned node
8              If the neighbour is not in the path that was just returned, then
9                  append the neighbour node to the current path, and append the
                       node and the updated path to the top of the stack
10     Return None
```

Code

```
1  def dfs(graph, start, end):
2      stack = [(start, [start])]
3      while stack:
4          node, path = stack[-1]
5          del stack[-1]
6          if node == end:
7              return path
8          for neighbor in graph[node]:
9              if neighbor not in path:
10                 new_index = len(stack)
11                 stack[new_index] = (neighbor, path + [neighbor])
12     return None
```

Which of the following statement(s) about the implementation above are **true**:

- ☐ The pop at line 4 is not correctly implemented, we must use the `pop()` method instead. That code will prevent the correct popping.
- ☐ On line 4 and 5 we cannot return and delete the index `-1` since it will raise an `IndexError` not having the index -1 in the list.
- ☐ On line 11 we cannot append to the list by using the index, we should use the `append()` method instead.
- ☐ This is not a depth-first search but a breadth-first search. In order to make it a DFS we should start from the bottom, which in this case is the `end` node.

Solution

☐ The pop at line 4 is not correctly implemented, we must use the `pop()` method instead. That code will prevent the correct popping.

☐ On line 4 and 5 we cannot return and delete the index `-1` since it will raise an `IndexError` not having the index -1 in the list.

☑ On line 11 we cannot append to the list by using the index, we should use the `append()` method instead.

☐ This is not a depth-first search but a breadth-first search. In order to make it a DFS we should start from the bottom, which in this case is the `end` node.

- The pop consists in returning the last item of a list and deleting it from the list itself. The `pop()` does it all in one method, in this case it's simply been re-implemented.
- You can indeed get negative indices out of an array, a negative index means that the index count should start from the end of the list. `[-1]` means the last item, `[-2]` means the second last, and so on.
- We cannot refer a position in the list that has never been initialized, since the stack didn't have the `new_index` position assigned (because it was `new_index` but indexing starts from 0, meaning the last index is `new_index-1`). Therefore we need to use the `append()` method to add an item at the end of the list.
- This is indeed a DFS algorithm since we always look at the current node's children first. We do not need to "start from the bottom" in order to have a DFS.

3.2 Task 2

The following code implements a Breadth-First Search, it simply looks whether it is possible to reach the destination node starting from the source node.

Pseudocode

```
Define bfs(graph, start, target)
    Create a set for the visited nodes
    Create a queue with the start node
    While there are nodes in the queue
        Set the current node to the first node in the queue
        If the current node is the target then return true
        Add the current node to the visited set
        For every neighbour of the current node
            If the neighbour is not in the visited set add it to the queue
    Return false
```

Code

```python
def bfs(graph, start, target):
    visited = set()
    queue = [start]

    while queue:
        current_node = queue.pop(0)
        if current_node == target:
            return True

        visited.add(current_node)

        for neighbor in graph[current_node]:
            if neighbor not in visited:
                queue.append(neighbor)

    return False
```

Which of the following statement(s) about the implementation above are **true**:

- [ ] The code will not work in case the graph is cyclical.
- [ ] The pop method does not have a parameter, it can just pop the last element.
- [ ] The method correctly implements a Breadth-First Search.
- [ ] The method would work only for directed graphs and not for undirected graphs.

Solution

☐ The code will not work in case the graph is cyclical.

☐ The pop method does not have a parameter, it can just pop the last element.

☑ The method correctly implements a Breadth-First Search.

☐ The method would work only for directed graphs and not for undirected graphs.

- The code works in both cyclical and acyclical graphs, nodes in cycles will not be visited twice since they are stored in the visited set and filtered at line 13.
- Even though the pop concept is derived from stacks where only the last element can be popped, in Python lists the pop method can take a parameter, which is the index of the element to remove and then delete. If no parameter is passed, the last item will be deleted just like in the classic pop concept.
- It does implement a BFS, it checks all nodes at the same depth before moving onto the deeper layer.
- Being directed or non directed is irrelevant since the edges would be represented in the edge list. In non directed graphs there will be an item in the list of two vertices instead of just one.

3.3 Task 3

The code computes the shortest distance between the `start_id` node and any other node in the graph. The algorithm used is Dijkstra's algorithm.

Pseudocode

```
Define dijkstra(graph, start_id)
    Create a distances dictionary, the values are the distance between the start_id node and the key (a node id)
    Create a dictionary to store the parent of vertices found along the shortest path
    Create a queue for vertices

    For every vertex in the graph
        Set its parent to -1
        Add the node to the queue
        Set the distance to infinity for all nodes, start_id distance set to 0


    While there are values in the queue
        Set the smallest known distance to infinity
        Set the node the distance refers to, to the next smallest node in the queue (later called the current node)

        The following for loop looks for the node with the shortest distance in the queue
        For every node in the queue
            If the distance between the start_id and the node is less than the smallest known distance, then set the current node to this node, together with its distance
        Remove the current node from the queue

        Now update the distances of the neihboring vertices passing by the current node
        For every neighbour of the current node that is still in the queue
            The distance from the neighbour to start_id passing by the current node is the distance of the current node from start_id + the distance of the current node and the neighbour
            If this new distance is lower than the previous distance of the neighbour from start_id
                Then update the distance in the distances dictionary
                In the parents dictionary, set the current node as the current neighbor parent

    Return the distances and parent dictionaries
```

Code

```
def dijkstra(graph, start_id: int) -> Tuple[dict, dict]:
    dist = {}
    parent = {}
    queue = []
    for v in graph:
        parent[v] = -1
```

```
7           queue.append(v)
8           dist[v] = float('inf') if v != start_id else 0
9
10      while queue:
11          u_val: float = float('inf')
12          u: Node = queue[0]
13          for q in queue:
14              u_val, u = (dist[q.id], q) if dist[q.id] > u_val else (u_val, u)
15          queue.remove(u)
16
17          children = [i for i in u.children if i in queue]
18          for v in children:
19              alt = v.weight
20              if alt < dist[v.id]:
21                  dist[v.id] = alt
22                  parent[v.id] = u.id
23      return dist, parent
```

Which of the following statement(s) about the implementation above are **true**:

- ☐ The ifs on line 8 and 14 are not a correct Python syntax.
- ☐ On line 14 a less than (`<`) should be used, since we want to get the node which distance is the shortest.
- ☐ On line 19, the new distance should be `dist[u.id] + v.weight` instead of only the last node weight (which is what we often call distance in this scenarios).
- ☐ This is a correct implementation of Dijkstra's algorithm.

Solution

☐ The ifs on line 8 and 14 are not a correct Python syntax.

☑ On line 14 a less than (`<`) should be used, since we want to get the node which distance is the shortest.

☑ On line 19, the new distance should be `dist[u.id] + v.weight` instead of only the last node weight (which is what we often call distance in this scenarios).

☐ This is a correct implementation of Dijkstra's algorithm.

- The syntax is indeed admitted, it is often called a ternary operator. It is just a shorthand for an if-else statement in python, just like in many other languages.
- It is true, if we need to find the closest node to `start_id`, we need to update `u` and `u_val` with a value that is less that the current distance, not greater.
- The distances in the `dist` dictionary represent the distances from the `start_id`, therefore we need to get the distance to the previous point (`u`) plus the distance between `u` and `v`, in order to get the whole path from `start_id` to `v`.
- Due to the 2 previous statements this is not a correct Dijkstra's algorithm implementation, by solving those two issues it would become one.

3.4 Questions

Which of the following statement(s) are true:

- [ ] When looking for an element in a tree we must perform both a BFS and a DFS since the BFS will look for the items at the same level of the starting point and the DFS will look for the items on the branch starting from the starting point. Using only one of them may exclude some items from the search.

- [ ] Dijkstra's algorithm can find the shortest path only for acyclical graphs, if cycles are in the graph then we need to use A* that will instead follow its heuristic without being tricked into the cycles.

- [ ] Dijkstra's algorithm runs on an unweighted graph (every edge has the same weight), works just like a classic BFS search.

3.5 Solution

☐ When looking for an element in a tree we must perform both a BFS and a DFS since the BFS will look for the items at the same level of the starting point and the DFS will look for the items on the branch starting from the starting point. Using only one of them may exclude some items from the search.

☐ Dijkstra's algorithm can find the shortest path only for acyclical graphs, if cycles are in the graph then we need to use A* that will instead follow its heuristic without being tricked into the cycles.

☑ Dijkstra's algorithm runs on an unweighted graph (every edge has the same weight), works just like a classic BFS search.

- BFS and DFS look for items within a tree. The way they operate is similar but they simply follow different paths, DFS explores the tree vertically first while BFS explores it breadth first. Regardless of the order they'll both check out every element, so using one of them is totally fine, we do not need to use both at the same time.
- Dijkstra's algorithm works also in cyclical graphs, since it will not revisit the exact same path twice. But it could visit the same node multiple times to try find a shorter path to it.
- Dijkstra's algorithm works just like BFS when no difference in weights between the edges is found. This is because Dijkstra is going to take the next node with the shortest distance from the source. Dijkstra is essentially a BFS with respect to the edge weights rather than the topology (BFS is guided by topology, not edge weights).